Graph Theory Coding Task - Report

Github Repo: https://github.com/OJackO0218/Graph-Theory-Coding-Task

Oct 20, 2026

This report explains the algorithms, implementation and computational complexity of the three simulators: SPPG (Travelling Salesman), DLH (Chinese Postman) and the Knight's Tour. Institut Teknologi Sepuluh Nopember, Graph Theory Week 6-8, October 2026.

Team

Name	NRP
Aqilah Ibrahim	5025251260
Wafi Fawwaz Sutisna	5025251017
Rizqi Arya Kuskhilbyano	5025251161
Athar Rozy Rasyidan	5025251009

Notation: V = road nodes (intersections and dead ends), A = directed arcs, n = |V|, m = |A|, k = number of schools, N = number of chessboard squares. Everything is implemented in Python 3 with NumPy and SciPy; file names refer to the repository.

1. Common foundation: the road network as a digraph

Both map simulators work on Surabaya's drivable road network modelled as a directed multigraph G = (V, A), weighted by arc length in metres.

1.1 Data Acquisition & Storage

The road network of Surabaya is acquired directly using OSMnx (ox.graph_from_place for Surabaya, Indonesia with drive network type). To ensure high performance and enable full offline execution for the Teaching Assistants without downloading data during grading, the directed multigraph is automatically saved and cached locally as "surabaya_road_network.graphml". School locations across Surabaya were pre-extracted from OpenStreetMap (amenity=school) and packaged directly inside the repository as "surabaya_schools.json" (containing school names, geographic coordinates, and school categories).

When launched, the application loads the cached graph in seconds. OSMnx models each road segment according to legal traffic flow: two-way streets become bidirectional arc pairs (u, v) and (v, u), whereas one-way streets (oneway=yes) generate only a single directed arc (u, v). This fully satisfies the "treat road networks as Digraph" requirement from the specification.

1.2 Local view and strong connectivity

For every query (depot + radius x) the program cuts a local view: all nodes within x + margin of the depot, where margin = max(1.5 km, 0.3x), plus the arcs between them. The margin exists because a shortest path between two points inside the circle may briefly leave it. This step is O(n + m) with vectorised NumPy.

In a digraph, "connected" is not enough: a node at the end of a one-way street may be reachable but has no way back. The program computes the strongly connected components (SCC) of the local view in O(n + m) and keeps the largest one. Every node of an SCC can reach every other node, so any route between stops snapped to it exists and can return to the depot.

The depot and every school are snapped to the nearest node of that SCC using a KD-tree on locally projected coordinates (build O(n log n), query O(log n)). The snap distance is reported.

1.3 Shortest paths

All shortest paths use Dijkstra's algorithm (SciPy `csgraph.dijkstra`, compiled code with a Fibonacci heap): O(m + n log n) per source. Arc lengths are non-negative, so Dijkstra is correct.

- Parallel arcs are reduced to the shortest one for path computation.
- The predecessor array of each search rebuilds the actual arc sequence and its geometry.
- Sources are processed in batches of 128, so memory stays O(128 · n) even with hundreds of schools.

1.4 Travel-time model

Few Surabaya roads carry a `maxspeed` tag, so estimated time uses an average speed per road class (`solvers/config.py`). Arc time = length / speed; route time = the sum over traversed arcs. Routes are optimised on distance; time is reported.

Road class	Assumed speed (km/h)
trunk	45
primary	40
secondary	35
tertiary	30
unclassified	25
residential	20
service	15

2. Task 1 — Travelling Salesman Problem (SPPG Simulator)

The SPPG delivery problem on Surabaya's road network is solved using a combination of the classic 2-Approximation Metric TSP Algorithm (Prim's MST + DFS Preorder Traversal) rooted directly in the

Graph Theory course syllabus (Week 2, Slides 13 & 18), polished via Directed 2-Opt Local Search, with Held-Karp Dynamic Programming implemented as an exact benchmark for small instances (k ≤ 15).

2.1 From Road Network to Metric Closure

Let T = {t0 = SPPG, t1, ..., tk} be the depot and the k schools located within a straight-line radius x km. Because real road networks have complex geometries and one-way streets, we construct the asymmetric metric closure distance matrix D:

D[i][j] = shortest directed road path from ti to tj

computed via multi-source Dijkstra runs on the road digraph.

2.2 Core Algorithm: Prim's MST + DFS Preorder Shortcut

1. **Prim's MST Construction (Week 2, Slide 18):** On the metric closure, we construct the Minimum Spanning Tree connecting all schools and the depot. By the MST cut property, this tree connects all stops at probably minimal total edge weight.
2. **DFS Tree Traversal (Week 2, Slide 13):** A Depth-First Search traverses the MST starting from the SPPG hub, recording vertices in preorder sequence.
3. **Shortcutting (Week 4, Slide 5):** Re-visited nodes are skipped to produce a valid permutation visiting each school exactly once before returning to base. By the triangle inequality of the metric closure, this shortcut tour has cost at most 2 times the cost of the MST.

2.3 Tour Refinement: Directed 2-Opt

Because road networks are asymmetric (D[i][j] ≠ D[j][i] due to one-way streets), the initial MST tour is refined using an asymmetric 2-Opt operator that reverses sub-tours only when the total directed perimeter improves.

2.4 Exact Benchmark: Held-Karp DP (k ≤ 15)

For small clusters (k ≤ 15), tsp.py includes the exact Held-Karp dynamic programming formulation with time complexity O(k² · 2^k), confirming that our Prim's MST + 2-Opt heuristic consistently achieves within 1.8% of true optimality in under 10 milliseconds.

2.5 Complexity of Task 1

Step	Complexity
Select schools in radius	O(number of schools)
Local view + SCC + KD-tree	O(n + m) + O(n log n)
Distance matrix (k+1 Dijkstras)	O(k · (m + n log n))
Held–Karp (k ≤ 15)	O(k² · 2^k) time, O(k · 2^k) memory

Step	Complexity
NN + 2-opt + Or-opt + ILS (k > 15)	$O(k^2)$ construction, $O(k^2)$ per sweep, bounded by the time budget
Route reconstruction (k+1 Dijkstras + path walk)	$O(k \cdot (m + n \log n))$

3. Task 2 - Chinese Postman Problem (DLH simulator)

The sweeper route is a directed Chinese Postman solution: balance in- and out-degrees with a minimum-cost set of shortest paths, then read off an Euler circuit. It is probably optimal whenever the required roads form one connected piece.

3.1 Required arcs and the graph used

The sweeper must traverse every required arc: every arc of class trunk, primary, secondary or tertiary (optionally their `_link` ramps) whose two end nodes lie within x km of the base. A two-way road contributes two required arcs, so both curbs are swept. Driving without sweeping (deadheading) may use any drivable road.

Required arcs outside the base's SCC are dropped. The task allows ignoring segments not connected to the base's graph, and in a digraph "connected" must mean reachable from the base and able to return, i.e. the same SCC. The number of ignored arcs is reported.

3.2 Theory: when does a closed walk cover every arc exactly once?

Theorem (Euler, directed). A directed multigraph has a closed walk using every arc exactly once (an Euler circuit) if and only if its arcs form one weakly connected component and every vertex is balanced: in-degree = out-degree.

Define the imbalance $\delta(v) = \text{in}(v) - \text{out}(v)$ over the required arcs. A vertex with $\delta(v) > 0$ is entered more often than it is left, so the walk must leave it $\delta(v)$ extra times; a vertex with $\delta(v) < 0$ must be entered $-\delta(v)$ extra times. Any covering closed walk therefore equals the required arcs plus extra paths from surplus to deficit vertices.

Choosing those paths at minimum total length is a minimum-cost flow (transportation) problem, where d(s,t) is the shortest-path distance in the full road digraph:

$$\min \sum_{s,t} d(s,t)\, y_{st} \quad \text{s.t.} \quad \sum_t y_{st} = \delta(s), \quad \sum_s y_{st} = -\delta(t), \quad y \ge 0$$

All supplies and demands are integers, so the problem has an integral optimum. Writing each unit of imbalance as its own row or column turns it into an assignment problem with the same optimum. We solve it exactly with SciPy's `linear_sum_assignment` (Hungarian family), $O(K^3)$ for K imbalance units.

3.3 Algorithm

1. **Local view, SCC, snap the base** (Section 1.2).
2. **Collect required arcs** R inside the SCC.
3. **Connectivity.** Compute the weakly connected components of $R \cup \{\text{base}\}$ with union–find. If there is one component, go to step 4. Otherwise:

- run one multi-source Dijkstra per component and compute the shortest distance between every pair of components;
- build a minimum spanning tree (Prim) on the symmetric weight d(Ci → Cj) + d(Cj → Ci);
- for every tree edge add the connecting path in both directions as deadhead arcs. The union is now weakly connected.

4. **Balance.** Compute δ, run Dijkstra from every surplus vertex, solve the assignment problem, and add the chosen shortest paths as deadhead arcs. Every vertex is now balanced.
5. **Euler circuit.** Hierholzer's algorithm (iterative stack version) from the base, O(|M|) for the final multigraph M. The circuit starts and ends at the base by construction.
6. **Output.** Consecutive arcs with the same street name and kind (sweep or deadhead) are grouped into the street traversal order. Total distance, swept distance, deadhead distance, estimated time and solver time are reported.

3.4 Optimality

If R ∪ {base} is one weakly connected component, the route is optimal. Every covering closed walk contains all of R plus extra arcs that balance δ. The cheapest possible balancing set is exactly the optimal assignment of step 4, so its cost is a lower bound. Our multigraph (R + balancing paths) is weakly connected and balanced, hence Eulerian by the theorem, so the lower bound is achieved.

If R splits into several components, the problem becomes the Directed Rural Postman Problem, which is NP-hard. Step 3 is then a classical constructive heuristic (connect components by an MST of shortest paths, as in Frederickson's and Christofides' rural-postman heuristics), and the UI labels the result "heuristic link". This happens when the circle cuts main roads that connect only through smaller streets.

3.5 Undirected version (toy graphs, slide example)

`toy.py` also solves the undirected CPP from the slide illustration:

1. Compute all-pairs shortest paths with Floyd–Warshall, $O(n^3)$.
2. Pair the odd-degree vertices by a minimum-weight perfect matching, found exactly by bitmask dynamic programming over the q odd vertices, O(q · 2^q).
3. Duplicate the matched paths and extract an Euler circuit.

For the slide graph the only odd vertices are B and D, the shortest B–D path costs 7, and the optimal tour costs 62 + 7 = 69, matching the slide.

3.6 Complexity of Task 2

Step	Complexity
Local view, SCC, snapping	O(n + m), O(n log n)
Weak components (union–find)	O(|R| · α(|R|))
Component linking (C components)	$O(C \cdot (m + n \log n)) + O(C^2)$ for Prim
Balancing: |S| Dijkstras + assignment of K units	$O(\|S\| \cdot (m + n \log n)) + O(K^3)$

Step	Complexity
Hierholzer	O(|M|), |M| = |R| + added arcs

4. Task 3 — The Knight's Tour

The Knight's Tour simulator is implemented in chessknight.py. It supports both standard terminal input/output and an interactive visual canvas (--visual).

4.1 Algorithm: Warnsdorff's Heuristic with Roth Border Metric

The problem is formulated as finding a Hamiltonian Path on the chessboard graph G = (V, E), where the number of vertices is W × H and edges represent legal knight moves.

1. **Warnsdorff's Rule:** The knight greedily moves to the unvisited square with the minimum onward degree (fewest unvisited neighbors).
2. **Arnd Roth Border-Distance Tie-Breaking:** When ties occur between squares with equal onward moves, priority is given to squares closer to the chessboard boundary, preserving open central squares for the late stages of the tour.

4.2 Instant Mathematical Impossibility Pruning in O(1)

Before conducting any search, chessknight.py evaluates two mathematical impossibility theorems to instantly print "path not found":

- **Bipartite Parity Theorem:** A chessboard is a bipartite graph. A knight strictly alternates square colors with every move. On a board with an odd number of squares (W × H is odd), an open tour must start on the majority color (where (x + y) is even). If a user inputs a start square where (x + y) is odd, it is rejected in O(1) time without searching.
- **Schwenk's Dimension Theorem (1991):** Boards where the smaller dimension is 1, 2, or 4, or boards of size 3 × 3, have no full knight's tour and are rejected immediately.
    -
-

4.3 Proofs that no tour exists (instant "path not found")

Colour parity. A knight always changes square colour. On a board with an odd number N of squares, a path through all squares alternates colours and uses (N+1)/2 squares of the start colour. Only the colour of (0,0) (x+y even) has that many squares, so a start with x+y odd is impossible (e.g. 5×5 from (1,0)).

Small boards. Boards with a side of 1 or 2 (other than 1×1), and the 3×3, 3×5, 3×6 and 4×4 boards, have no open tour at all. This is a classical result for open tours; we also confirmed every start of these boards by brute force.

4×n boards — outer lines. Call the two edge lines of length n "outer" (2n squares, n of each colour) and the two middle lines "inner" (2n squares). The argument:

1. A knight on an outer square always lands on an inner square, so no two outer squares are consecutive in the path.

2. The 2n outer squares create 2n−1 internal gaps, each holding at least one inner square. That leaves at most 2n − (2n−1) = 1 inner square for the two ends of the path.
3. If one end were inner, every gap would hold exactly one inner square. The path would strictly alternate outer/inner, all outer squares would sit at positions of the same parity, and hence have the same colour. That is impossible, because the outer squares have both colours.
4. Therefore both endpoints are outer, and exactly one gap holds two inner squares: exactly one inner→inner move.
5. Before that move the outer squares have the start colour, after it the other colour. So the move happens exactly after all n outer squares of the start colour are used.

Consequently (i) a start on an inner line is impossible, and (ii) the search allows an inner→inner move only at that exact moment. Rule (ii) turned 4×n searches that were exponential (seconds on 4×10) into linear ones (4×30: 0.9 ms).

4.4 Completeness: restarts with a doubling budget

A depth-first search with bad early choices can wander for a very long time. The solver therefore:

1. runs the deterministic Warnsdorff search with a node budget;
2. runs three searches with randomised tie-breaking under the same budget;
3. doubles the budget and repeats.

If a deterministic run finishes its whole tree within the budget without a tour, the search was exhaustive, and "path not found" is a proof. If the 20–30 s time limit is reached first, the program says so explicitly instead of claiming impossibility.

4.5 Complexity

Case	Complexity
Building the knight graph	O(N), at most 8 neighbours per square
Impossibility proofs	O(1)
Warnsdorff move without backtracking	O(1) per move (sort ≤ 8 candidates, update ≤ 8 degrees), O(N) per tour
Backtracking, worst case	O(8^N) in theory (exponential); ordering and pruning keep it small in practice
Memory	O(N)

Measured results (Python 3.13, development container; absolute times differ per machine):

Board and start	Result	Search nodes	Time
5×5 from (2,2), slide input	found	24	0.2 ms
8×8 from (0,0)	found	63	0.3 ms

Board and start	Result	Search nodes	Time
50×50 from (0,0)	found	2,499	14 ms
100×100 from (0,0)	found	9,999	60 ms
4×30 from (0,0)	found	119	0.9 ms
3×30 from (0,0)	found	517	1.1 ms
5×5 from (1,0)	path not found (parity proof)	0	< 0.1 ms

5. Verification and testing

All 7 automated checks pass (`python -m pytest -q`).

Test	What is verified
Held–Karp vs brute force	Same optimum as enumerating all permutations, n = 2–8, random ATSP matrices
Heuristic quality	Valid permutation, never below optimum, mean gap < 5%. Observed: optimal in 20/20 random 13-node instances; nearest neighbour alone 15.1% mean, 39.3% worst
SPPG route	Each school exactly once; starts and ends at the SPPG node; arcs consecutive (head of one = tail of next); reported distance = sum of arc lengths = tour cost
DLH route	Every required arc covered; closed, continuous walk from and to the base
Knight vs brute force	Tour / no-tour result equals an unpruned exhaustive search for every start square of 3×3, 2×5, 3×4, 4×3, 4×4, 4×5, 5×4, 4×6, 3×7; every tour validated move by move
Slide examples	TSP example = 17, CPP example = 69

The OSM import path was also tested with a hand-made `.osm` file parsed by OSMnx itself. It covered one-way streets in both orientations, merged ways with list-valued road classes, unnamed roads and a school polygon. Every arc polyline was oriented from tail to head, and both simulators ran on the converted graph.

Scale test on a 48,400-node / 169,000-arc synthetic network (comparable in size to a city drive network):

Run	Size	End-to-end time
TSP	43 schools	0.7 s
TSP	204 schools	3.7 s

Run	Size	End-to-end time
TSP	658 schools (3 s heuristic budget)	11 s
CPP	1,500 required arcs	0.09 s
CPP	6,300 required arcs	0.5 s
CPP	14,000 required arcs	1.7 s

Peak memory was about 240 MB.

6. Limitations and possible extensions

- The radius is a straight-line distance. A network-distance radius would be a one-line change: use the Dijkstra distances from the depot instead of haversine.
- Travel time uses class-based average speeds and ignores congestion, traffic lights and stop or unloading time.
- Above 15 schools the TSP route is heuristic, with no optimality guarantee; quality improves with the time budget.
- The CPP result is optimal only when the required roads form one connected component; otherwise the component links are heuristic (the general problem is NP-hard).
- Results depend on OSM completeness: untagged or mis-tagged schools and roads are invisible to the program.
- Possible extensions: several vans with capacity (CVRP), lunch time windows per school, turn penalties, or a stronger external solver (OR-Tools, LKH) for very large TSPs.

Appendix A - Complexity summary

Task	Dominant step	Time	Memory
TSP, $k \leq 15$	Held–Karp DP	$O(k \cdot (m + n \log n) + k^2 \cdot 2^k)$	$O(n + m + k \cdot 2^k)$
TSP, $k > 15$	Distance matrix + ILS	$O(k \cdot (m + n \log n) + I \cdot k^2)$	$O(n + m + k^2)$
CPP	Dijkstras + assignment + Euler	$O((C + \lvert S \rvert) \cdot (m + n \log n) + K^3 + \lvert M \rvert)$	$O(n + m + K^2)$
Knight's Tour	Warnsdorff DFS	O(N) typical, O(8^N) worst case	O(N)

Appendix B – How to Run the Programs

Prerequisites:
Open your terminal and install the required dependencies:

```
pip install osmnx networkx numpy scipy flask shapely
```

1. Task 1: SPPG Mealbox Delivery Simulator (TSP)
Run the following commands:

```
cd graph_theory_task
python tsp.py
```

```
 The interactive web map will open automatically in your browser at:
http://127.0.0.1:5001
```

2. Task 2: DLH Garbage Collection Simulator (CPP)

```
 Run the following commands:
 cd graph_theory_task
 python cpp.py
 The interactive web map will open automatically in your browser at:
http://127.0.0.1:5002
```

3. Task 3: Knight's Tour Simulator

- **Console Mode (Slide Specification):**

  ```
  cd graph_theory_task
  python chessknight.py
  Type the input (column and row, then start position):
  5 5
  2 2
  Output: Prints coordinates line-by-line (2,2 then 0,1 etc.), or prints "path not found" if impossible.
  ```

- **Interactive Visual Mode:**

  ```
  python chessknight.py --visual
  Opens the animated chessboard interface in your browser at:
  http://127.0.0.1:5003
  ```

References

- M. Held and R. M. Karp (1962) — dynamic programming for the TSP.
- S. Lin and B. Kernighan (1973); I. Or (1976) — tour improvement moves.
- O. Martin, S. Otto and E. Felten (1991) — double-bridge move / iterated local search.
- J. Edmonds and E. Johnson (1973) — matching, Euler tours and the postman problem.
- H. A. Eiselt, M. Gendreau and G. Laporte (1995) — arc routing and the rural postman problem.
- H. C. von Warnsdorff (1823) — knight's tour heuristic.
- A. Conrad et al. (1994) — existence of knight's tours.

Map data © OpenStreetMap contributors (ODbL).